Brickts SQL course — Week 7 — Build defensible fault detection

sparql-playground / lessons/sql

Generated for offline / Kindle reading

Week 7 — Build defensible fault detection

This pack concatenates the six daily lessons and the weekly challenge, then the Week primary-reading links from SOURCES.md. Work the SQL in `psql` against `brickts_course`; keep notes under ignored `student-work/sql/`.

Contents

- Day 43: Day 43 — Express a fault predicate without turning unknown into healthy
- Day 44: Day 44 — Confirm consecutive candidates with gap-aware islands

- Day 45: Day 45 — Gate rules by mode and allow startup time
- Day 46: Day 46 — Bound the rule window and retrieve enough context
- Day 47: Day 47 — Separate rule applicability from per-equipment tuning
- Day 48: Day 48 — Preserve raw evidence beside confirmed fault summaries
- Day 49 (weekly challenge): Day 49 — Week 7 independent challenge — a known-answer pressure FDD report

---

Day 43 — Express a fault predicate without turning unknown into healthy

Previous: Day 42 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–42, or equivalent SQL skills.

Goal

Translate a small pressure rule into SQL while preserving off, unavailable, pass and candidate states.

Before you start

Use `course.ahu_5m`; it aligns roles by equipment and instant without filling missing inputs. Review the full Open-FDD distinction in the bridge notes. Today's SQL is a teaching predicate, not the complete production FC1 algorithm.

Concept

Our candidate condition is `duct_static < 0.9 * duct_static_sp` while valid `fan_status = 1`. Pressure and setpoint are Pa. Require eligible inputs, binary status and positive setpoint. Fan off is excluded from evaluation. Missing inputs while the fan is on produce unknown, not a pass. Good data that fails the candidate condition produces evaluated false.

SQL has true, false and null/unknown. `WHERE predicate` retains only true rows, discarding false and unknown alike. `COALESCE(predicate, false)` can erase a data-quality problem. Preserve an evaluation-state column before filtering candidates. The alternative threshold `static < setpoint - 50 Pa` happens to classify this fixture identically, but is a different rule for other setpoints.

Tiny example

```
SELECT measured_pa,
       measured_pa < 0.9 * 250.0 AS
         raw_candidate
FROM (VALUES (100.0), (225.0), (260.0),
         (NULL)) AS v(measured_pa);
```

Results are true, false, false and null. Equality at the threshold does not satisfy a strict `<` comparison.

SQL challenge

Produce a bounded evaluation detail table for every equipment and timestamp. Include pressure, setpoint, fan state, input validity, evaluation state and nullable candidate Boolean. Summarize evaluated-true, evaluated-false, off and unknown time separately using the interval policy.

Experiment

Compare `WHERE raw_candidate`, `WHERE NOT raw_candidate` and `WHERE raw_candidate IS NULL` on your scratch values. Add fan-off rows with missing pressure and document your state precedence before querying.

Acceptance checks

- North Supply A has ten raw candidate samples on Jan 5, with two unknown pressure instants inside the planted excursion.
- The other equipment have no raw pressure candidates.
- Off rows are labelled excluded; missing on-state pressure never becomes a healthy pass.
- A 225 Pa measurement at a 250 Pa setpoint is false under the strict teaching predicate.

Stretch and reflection

What evidence does a candidate provide about cause? Explain why a low-pressure condition alone cannot prove a failed fan or a sensor fault.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day43/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 42 ·

Day 44 — Confirm consecutive candidates with gap-aware islands

Previous: Day 43 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–43, or equivalent SQL skills.

Goal

Require sustained evidence without joining observations across a missing interval.

Before you start

Retain Day 43's complete evaluation stream, including false and unknown rows. Today's convention confirms the third consecutive eligible true observation, then later true observations in that same run. It does not backdate confirmation to the first candidate.

Concept

A run starts when the current row is true and the preceding row is not eligible true, or when the timestamp gap exceeds 300 seconds.

Partition by site, equipment and rule. Use `LAG` to inspect the prior timestamp and prior state, then derive run boundaries and an island identifier in separate query stages. Count observations within each true island.

Filtering to true rows before identifying breaks can join candidates separated by normal or unavailable data. Gap checks also matter when an entire pivot row is absent. Three consecutive observations are a count-based policy; they do not automatically mean fifteen elapsed minutes of evidence. Here observations are five minutes apart and intervals are at most five minutes.

Tiny example

```
SELECT ts, extract(epoch FROM ts - lag(ts)
         OVER (ORDER BY ts)) AS
         gap_seconds
FROM (VALUES ('2026-01-05
         08:00Z'::timestamptz),
             ('2026-01-05
         08:05Z'::timestamptz),
             ('2026-01-05
         08:15Z'::timestamptz)) AS v(ts);
```

The last gap is 600 seconds, so a true run must restart even if every shown row is a candidate.

SQL challenge

Add island identity, position within the true island and a confirmed flag to your pressure detail. Show both raw and confirmed interval seconds. Keep quality breaks and time-gap breaks auditable rather than storing only the final Boolean.

Experiment

Use scratch candidates at 00:00, 00:05, 00:15, 00:20 and 00:25. Predict the first confirmed timestamp. Repeat with an explicit unknown row at 00:10 instead of an absent row.

Acceptance checks

- Fixture candidates form two five-observation islands, separated by the pressure gap.
- Confirmed fixture starts occur at Jan 5 08:10Z and 08:45Z.
- Raw candidate time is 3,000 seconds; confirmed time is 1,800 seconds under this non-backdating convention.
- Scratch confirmation begins at 00:25; inserting an unknown break cannot increase confirmation.

Stretch and reflection

Compare three observations with a ten-minute persistence timer. Under irregular cadence, what different events could each policy confirm?

Save SQL, actual output and a short interpretation in your own `student-work/sql/day44/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 43 ·

Day 45 — Gate rules by mode and allow startup time

Previous: Day 44 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–44, or equivalent SQL skills.

Goal

Exclude expected startup behavior without using a fixed clock time as a substitute for operating history.

Before you start

The fixture has no general heating/cooling operating-mode column. Today uses a declared fan-on evaluation mode and synthetic scratch mode labels. Keep those labels separate from measured fixture roles.

Concept

Rule applicability can change with operating mode. A supply-air cooling rule needs a cooling context; an economizer rule may need

adequate outdoor/return temperature separation and an enabled economizer. Filling absent mode information with a convenient default invents evidence.

For the pressure exercise, add a ten-minute fan startup delay. Derive continuous on history from observed status transitions and break it on off, unknown or an excessive gap. Evaluate only when at least ten minutes of continuous eligible on evidence are known. At the start of a dataset, an already-on fan has an unknown actual start; a sufficiently long observed on stretch can establish a lower bound on runtime. Record that convention.

Tiny example

```
SELECT now_ts - startup_ts >= interval '10
         minutes' AS warmup_complete
FROM (VALUES ('2026-01-05
         06:10Z'::timestamptz,
              '2026-01-05
         06:00Z'::timestamptz)) AS
         v(now_ts, startup_ts);
```

At exactly ten minutes, the stated `>=` policy allows evaluation. That boundary choice belongs in rule metadata.

SQL challenge

Add mode eligibility, continuous-on age or lower bound, warmup eligibility and exclusion reason to your pressure detail. Preserve the raw teaching comparison separately from the mode-gated candidate. Confirm candidates only after applying the gate.

Experiment

Create a scratch startup with low pressure for the first ten minutes and normal pressure afterward. Compare results with and without the gate. Add one unknown status interval in the middle; predict how it affects your known continuous-on history.

Acceptance checks

- North Supply A's Jan 5 06:00Z startup is excluded until 06:10Z under the declared policy.
- Its 08:00Z pressure excursion remains eligible; the warmup gate does not erase it.
- Unknown startup history is visible rather than guessed from a schedule.
- Scratch missing mode or status cannot create a confirmed fault.

Stretch and reflection

How could repeated short fan cycles interact with this delay? Describe a separate cycling metric without weakening the pressure rule's declared startup policy.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day45/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 44 ·

Day 46 — Bound the rule window and retrieve enough context

Previous: Day 45 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–45, or equivalent SQL skills.

Goal

Evaluate a requested reporting window consistently while retrieving only the finite history its policy requires.

Before you start

Reuse the 300 second interval cap, three-observation confirmation and ten-minute startup delay. Distinguish the requested output window from the earlier input context used for computation.

Concept

Filtering inputs to the reporting start before computing windows changes the first results. A confirmed candidate at 08:10 may depend

on candidates at 08:00 and 08:05. Startup state and interval clipping also need preceding observations. Retrieve bounded context, evaluate in timestamp order, and trim output only after deriving state.

For these course policies, a conservative twenty-five-minute lookback covers ten minutes to establish warmup eligibility at the earliest needed candidate, ten minutes of confirmation history, and a possible five-minute predecessor for an arbitrary start inside an interval. This bound supports current eligibility; it does not promise reconstruction of the fan’s true startup time when it began hours earlier. Avoid future observations when defining a causal confirmation flag. A next timestamp may bound interval coverage without providing future candidate evidence.

Tiny example

```
SELECT report_start - interval '25
         minutes' AS context_start
FROM (VALUES ('2026-01-05
         08:12Z'::timestamptz)) AS
         v(report_start);
```

The acquisition start is 07:47Z, but reported intervals still begin no earlier than 08:12Z.

SQL challenge

Parameterize a bounded pressure report with start and end instants. Evaluate confirmation and startup from finite lookback, include the sample governing the start if eligible, and clip durations to the requested half-open window. Reject an end that is not later than start. Keep SQL values bound when called from a program.

Experiment

Compare `[08:10Z,08:20Z)` with `[08:12Z,08:18Z)` on Jan 5 for north Supply A. Then run a longer enclosing window and compare its clipped detail with the short-window result. Predict equality before executing.

Acceptance checks

- The aligned ten-minute window has 600 confirmed seconds; the inside-interval six-minute window has 360.
- Context rows never inflate the output duration.
- Short-window confirmation agrees with the same instants from the enclosing report.
- SQL has explicit bounds and does not scan unlimited history to infer startup.

Stretch and reflection

If confirmation changes to ten observations or an hour-long delay, which lookback bound must change? Record that relationship alongside the policy version.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day46/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 45 ·

Day 47 — Separate rule applicability from per-equipment tuning

Previous: Day 46 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–46, or equivalent SQL skills.

Goal

Check whether a rule has suitable inputs before applying an equipment-specific threshold policy.

Before you start

Use equipment UUIDs from `course.equipment`, not labels. All starter AHUs have the teaching pressure roles, but future equipment or a changed graph may not. Do not assume that successful evaluation on one AHU proves fleet applicability.

Concept

Applicability asks whether equipment kind, roles, units and selection policy support a rule. Tuning chooses thresholds, delays and confirmation settings for applicable equipment. Missing roles are an applicability result; missing samples during an otherwise applicable window are data availability. Keep those reasons distinct.

Represent tuning as data keyed by equipment identity and rule version, with units, effective bounds and provenance. A threshold from one AHU should not leak to another equipment that shares its label. Conflicting effective policy rows can multiply evaluations just like a faulty sensor join. Validate one selected policy per equipment and instant; do not quietly take the first row.

Tiny example

```
SELECT equipment_id, role, count(*) AS
         matching_points
FROM course.point
WHERE role IN ('duct_static',
         'duct_static_sp', 'fan_status')
GROUP BY equipment_id, role
HAVING count(*) <> 1;
```

The starter fixture returns no rows. This detects duplicates among present roles, but cannot detect an entirely absent role without an expected-role relation.

SQL challenge

Create scratch rule-requirement and equipment-policy relations. Audit each AHU for required roles, compatible units and exactly one selected point. Give policies a pressure ratio, warmup seconds and confirmation count with validated ranges. Demonstrate both a missing role and duplicate policy using scratch metadata.

Experiment

Give north and south Supply A different scratch ratios while retaining the same labels. Confirm each receives only its own policy. Then create overlapping effective rows for one equipment and make your audit reject ambiguity before evaluation.

Acceptance checks

- Starter pressure applicability succeeds for three distinct equipment UUIDs.
- Missing role, ambiguous role selection, incompatible unit and missing sample

have different explanations.
- Overlapping policies cannot silently double fault seconds.
- A tuning change records a policy identity and is described as a changed rule configuration.

Stretch and reflection

Would a ratio threshold transfer better than an absolute Pa threshold? Explain what equipment knowledge would still be required before choosing either one.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day47/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 46 ·

Day 48 — Preserve raw evidence beside confirmed fault summaries

Previous: Day 47 · **Week 7 · 45–75 minutes.** Prerequisites: Days 1–47, or equivalent SQL skills.

Goal

Design fault detail and summaries that a reviewer can trace back to input measurements and time policies.

Before you start

Reuse your pressure predicate, startup gate and confirmation convention. Read the distinction between teaching SQL and production rule behavior in the bridge notes.

Concept

Raw candidate, confirmed condition, confirmed duration and event count are different outputs. A Boolean at one timestamp is not an event count. Summing Boolean

values counts observations; summing bounded interval seconds measures duration. Two separate confirmed runs can have the same total seconds as one longer run.

Retain detail at equipment/rule/evaluation-time grain: input values and validity, state, threshold, raw Boolean, exclusion reason, run identity, confirmation position, confirmed flag, interval seconds and policy identity. A summary can then aggregate counts and seconds without losing the ability to explain a surprising result. Name unavailable and excluded duration separately from evaluated false duration.

Tiny example

```
SELECT count(*) FILTER (WHERE confirmed)
         AS confirmed_observations,
       sum(seconds) FILTER (WHERE
         confirmed) AS confirmed_seconds
FROM (VALUES (true, 300), (true, 120),
         (false, 300)) AS v(confirmed,
         seconds);
```

The results are two observations and 420 seconds. Neither number alone says how many events occurred.

SQL challenge

Create separate scratch detail and summary views for your bounded pressure report. Summarize raw seconds, confirmed seconds, confirmed island count, unknown seconds and eligible coverage by site/equipment/rule. Preserve enough keys to select all detail for one event without matching by equipment label.

Experiment

Compare two synthetic datasets with identical confirmed seconds: one continuous run and two separated runs. Demonstrate that event counts differ. Then shorten the final interval and show that duration changes while the number of confirmed observations may not.

Acceptance checks

- Fixture north Supply A has 3,000 raw seconds and 1,800 confirmed seconds across two confirmed islands under the three-observation policy.
- The ten-minute pressure gap is unknown, not an event or a pass.
- Summary duration exactly reconciles to detail interval seconds.

- Output says `confirmed_seconds`; it does not label that number simply `faults`.

Stretch and reflection

Which detail would you include in an operator-facing explanation? Choose a compact evidence set that supports review without implying the rule knows the physical root cause.

Save SQL, actual output and a short interpretation in your own `student-work/sql/day48/` directory. Label evidence **observed**, **fixture-only**, or **not run**. Consult this week's primary references for exact language and domain details.

Previous: Day 47 ·

Day 49 — Week 7 independent challenge — a known-answer pressure FDD report

Previous: Day 48 · **Week 7 review · 2–4 hours, split across sessions as needed.** Prerequisites: Days 43–48, or equivalent skills.

Project brief

Independently build a bounded SQL pressure-condition report for all fixture AHUs. Use eligible fan-on status, positive pressure setpoint and `duct_static < 0.9 × duct_static_sp`. Apply a ten-minute startup delay. Confirm the third consecutive eligible true observation, break on false, unknown or a gap over 300 seconds, and do not backdate confirmation. Use bounded left-held intervals clipped to the window.

This is a declared teaching rule. It does not claim to reproduce the complete current Open-FDD FC1 implementation.

Acceptance criteria

- Full-window north Supply A has ten raw candidate observations, two unavailable pressure instants within the excursion and two candidate islands.
- Raw duration is 3,000 seconds; confirmed duration is 1,800 seconds in two confirmed islands. Other equipment have no confirmed pressure condition.
- Off, warmup, unknown and evaluated-false states remain distinguishable.
- The short-window result agrees with an enclosing report after clipping, including starts between sample instants.
- Per-equipment applicability and a unique effective policy are checked before evaluation.

Deliverables

Provide SQL, detail and summary outputs, a policy card and two scratch counterexamples covering a gap and a threshold boundary. Explain one way an apparently reasonable query could overstate confirmation or hide unavailable evidence.

Self-review

Can every summary second be traced to detail? Which decisions came from SQL semantics, and which were domain policy? What additional adapter and catalog checks are required before comparing this output with production Open-FDD?

Label results **observed**, **fixture-only**, or **not run**. This page intentionally provides no worked solution or implementation sequence. Attempt the challenge independently before consulting the week's references.

Previous: Day 48 ·

Primary reading — Week 7

Use the lesson first, then these references for precise behavior. Full list: SOURCES.md.

Week 7

- Window functions
- SQL Boolean logic
- Open-FDD FC1 and runtime sources

- SQL expression syntax